

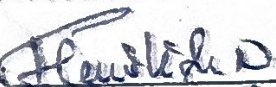
SINATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 1 - ANALYTICAL PROBLEM SOLVING

Course Code	CSA402	Course Title:	COMPILER DESIGN
CPI Assessment	CPI - Imitation (L1)	Assessment:	Assessment Tool 1 - ANALYTICAL PROBLEM SOLVING
Weightage	40% of CIA		
CPI Statement	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name	HARI VISHNU N	Reg No.:	192524319
Section/ Batch	AI & DS / 2nd year	Date:	16/07/2026

Code of Conduct

(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student


Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Sl. No.	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	3/4	3/4	4/4	4/4	3/4	86.25
2	4/4	3/4	4/4	3/4	4/4	90
3	3/4	4/4	3/4	4/4	4/4	88.75
4	4/4	4/4	3/4	4/4	4/4	93.75
5	3/4	3/4	4/4	4/4	4/4	90
OVERALL RUBRICS VALUE						
	/4	/4	/4	/4	/4	89.75
	/20	/20	/25	/20	/15	/100



Annexure A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression $P = (a * b) + (b * c) * 2$ through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a) $(\epsilon aa^*)^*$ b) $(a^* b^*)^* abb$ c) $(a^* b^*)^* aba (a^* + b^*)^*$ d) $(a ab)^*$ e) $(aba)^* + (a^* b^*)^*$
3	i) Find the number of tokens in the following C statement is <pre>main() { int a=10; char b="abc"; in t c = 30; ch ar d = "xyz"; in /* comment */ t m = 40.5; }</pre> ii) Identify the lexical errors in the following C statement is <pre>void main() { int x=10, y=20; char * a; a= &x; x= 1xab; }</pre>
4	Find a regular expression corresponding to each of the following subsets of $\{0 1\}^*$: i) The language of all strings that have an even number of 0's. ii) The language of all strings that contain at least one 1.

iii) The language of all strings in which both the number of 0's and the number of 1's are odd.

iv) The language of all strings that start with 1 and end with 0.

v) The language of all strings that do not contain consecutive 1's.

5 Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, ;, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string:

`int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }`

Questions:

- a) How many tokens are there in the given input string?
- b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).
- c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

$$P = (a * b) + (b * c) * 2$$

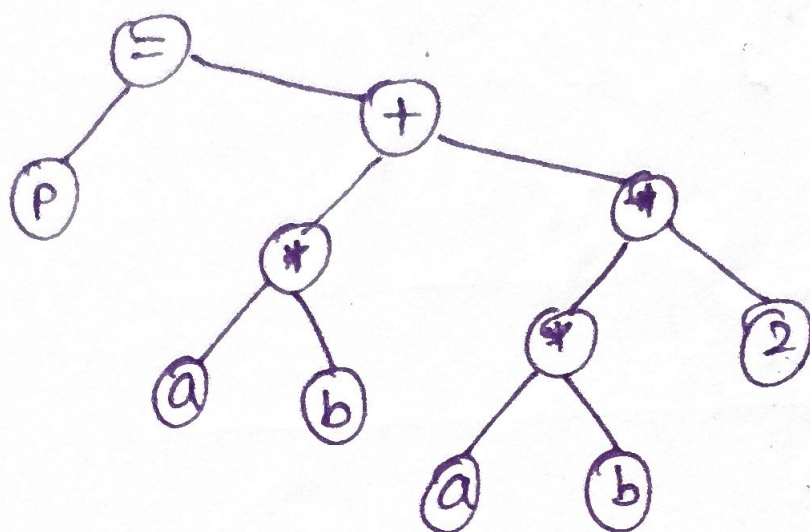
Lexical analysis

Lexeme	Token
P	ID
=	assignment operator
(	left parenthesis
a	ID
*	Multiplication operator
b	ID
)	Right parenthesis
+	left parenthesis
(	addition parenthesis
b	ID
*	Multiplication operator
c	ID
)	Right parenthesis
*	Multiplication operator
2	constant (NUM).

Token sequence :

$$ID(P) = (ID(a) * ID(b)) + (ID(b) * ID(c)) * NUM(2)$$

Syntax Tree



Intermediate code :

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$t_4 = t_1 + t_3$$

$$P = t_4$$

Code optimization

Note :

$$t_1 = a * b$$

$$R_4 \rightarrow a * b$$

$$t_2 = b * c$$

$$R_3 \rightarrow (b * c) * 2$$

$$t_3 = t_2 * 2$$

$$R_2 \rightarrow 2$$

$$P = t_1 + t_3$$

Target code :

MOV R4, a

MUL R4, b

MOV R3, b

MUL R3, c

MOV R2, 2

MUL R3, R2

ADD R4, R3

MOV P, R4

$$(\epsilon | aa^*)^* \quad (P).$$

$$ii) (a^* + b^*)^* abb$$

$$iii) (a^* b^*)^* aba (a^* + b^*)^*$$

$$iv) (a + ab)^*$$

$$v) (aba)^* + (a^* b^*)^*$$

$$(\epsilon + aa^*)^*$$

$$aa^* = a^+$$

$$(\epsilon + aa^*)^* = a^*$$

Five Valid string:

ϵ

a

aa

aaa

aaaa

$$(a^* + b^*)^* abb$$

all str over a, b that ends with abb.

Five valid string:

abb

aabb

babb

aaabb

bbabb

iii) $(a^* b^*)^* aba (a^* + b^*)^*$

iv) All str over $\{a, b\}$ that contain aba

aba

aaba

abab

abaa

babab

v) $(a + ab)^*$

Zero or more repetitions of a or ab.

Five valid strings:

ϵ

a

ab

aa

aab

vi) $(aba)^* + (a^* b^*)^*$

Since $(a^*)^* = (a + b)^*$

$(aba)^* \neq (a + b)^* = (a + b)^*$

Five valid strings

ϵ

a

b

ab

aba

$$(\epsilon / aa^*)^* \quad (P)$$

$$ii) (a^* + b^*)^* abb$$

$$iii) (a^* b^*)^* aba (a^* + b^*)^*$$

$$iv) (a + ab)^*$$

$$v) (aba)^* + (a^* b^*)^*$$

$$i) (\epsilon + aa^*)^*$$

$$aa^* = a^+$$

$$(\epsilon + aa^*)^* = a^*$$

Five Valid string:

ϵ

a

aa

aaa

aaaa

$$ii) (a^* + b^*)^* abb$$

all str over a, b that ends with abb.

Five valid string:

abb

aabb

babb

aaaaabb

bbabb

$$= N_1) (a^* b^*)^* aba (a^* + b^*)^*$$

Q) All str over $\{a, b\}$ that contain aba

aba

aaba

abab

abaa

babab

$$R) (a + ab)^*$$

Zero or more repetitions of a or ab

five valid strings:

ϵ

a

ab

aa

aab

$$U) (aba)^* + (a^* b^*)^*$$

Since $(a^*)^* = (a + b)^*$

$$(aba)^* \neq (a + b)^* = (a + b)^*$$

five valid strings

ϵ

a

b

ab

aba

i) Finding the number of tokens:

Line	Statement	Tokens	No. of Tokens.
1	main()	main, (,)	3
2	{	{	1
3	int a = 10;	int, a, =, 10, ;	5
4	char b = "abc";	char, b, =, " , abc, " , ;	5
5	int t c = 30;	int, c, =, 30, ;	5
6	char d = "xyz";	char, d, =, " , xyz, " , ;	5
7	/*comment*/ tm = 40.5;	int, m, =, 40.5, ;	5
8	}	}	1

Total no. of tokens: 30

ii) Identify the lexical errors:

```

void main() {
    int x = 10, y = 20;
    char *a;
    a = &x;
    x = 1xab;
}
    
```


Statement :

$X = 1xab;$

~~lexical~~

lexical error:

$1xab$

Invalid numeric constant. Hexadecimal number must begin with 0x.

Correct form:

$X = 0xab;$

4. i) Language: all string having an even number of 0's.

Regular expression: $1^*(01^*01^*)^*$

- ii) Language: all string containing at least one 1.

Regular exp: $(0+1)^*1(0+1)^*$

- iii) Language: all string in which both the number of 0's and 1's are odd.

RE: $1^*(01^*01^*)^*01^*(10^*10^*)^*$

- iv) Language: all string that start with 1 and end 0

Regular exp: $1(0+1)^*0$

v) language:

all string that do not contain
consecutive 's'.

Regular

exp: $(0+10)^*(1+\epsilon)$

Given Input:

Int var123 = 12345;

float var456 = 123.45;

If (var123 == var456)

{ return ; }

1) Total no. of Tokens:

S.NO	Token	Type
1	Int	keyword
2	var123	identifer
3	=	operator
4	12345	Integer
5	;	separator
6	float	keyword
7	var456	identifer
8	=	operator
9	123.45	floating
10	;	separator
11	If	keyword
12	{	separator
13	var123	identifer
14	==	operator
15	var456	identifer
16	}	separator
17	{	"
18	return	keyword
19	;	separator
20	}	"

2) Token categorized by Type:

Keyword - Put, float, pf, return

operator - =, ==

Separator - ;, :, (,), {, }

Identifier - var123, var456

Integer literal - 12345

Float literal - 123.45

8) Estimated FSM state Transition

Token type	no. of Token	Apx. state Trans. per token	Total Transitions
Keyword	4	2	8
Identifier	4	3	12
Int literal	1	2	2
Float literal	1	4	4
Operator	3	2	6
Separator	7	1	7

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/for mula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited unclear interpretation or	No interpretation